ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №8

«Мережеве та асинхронне програмування. REST API»

з курсу «Професійний Python»
Варіант №3 — «Інтернет-магазин»

Виконав(ла): Вересюк Ірина
Група: ФЕП-32
Перевірив(ла): Куньо І.М.
Дата: 01.11.26

Львів — 2026

1. Мета роботи

Ознайомитися з основами мережевого програмування у Python, принципами протоколу HTTP, архітектурним стилем REST і форматом JSON. Навчитися створювати REST API за допомогою FastAPI, описувати та перевіряти дані за допомогою Pydantic, використовувати async/await та asyncio, виконувати декілька операцій паралельно через Task і gather(), обмежувати час виконання (timeout), реалізовувати повторні спроби (retry), обмежувати кількість одночасних операцій через Semaphore та працювати з бібліотекою HTTPX.

2. Індивідуальний варіант і постановка задачі

Лабораторна робота №8 «Мережеве та асинхронне програмування. REST API», варіант №3 — «Інтернет-магазин». Ресурси: products, categories.

Необхідно реалізувати асинхронний REST API з endpoints: GET /products; GET /products/{id}; POST /products; PATCH /products/{id}; DELETE /products/{id}; GET /categories/{id}/products; POST /products/{id}/purchase. Функціонал: додавання товару, пошук, фільтрація за категорією, зміна ціни та кількості, видалення, покупка товару (транзакційна логіка), визначення найдорожчого товару, загальна вартість залишків. Асинхронне завдання: паралельно отримувати дані про декілька товарів із використанням Task, gather, Semaphore, HTTPX, timeout і retry.

Загальна архітектура: HTTP Client → FastAPI REST API → Pydantic → Service Layer → Repository Layer → SQLAlchemy / SQLite.

3. Структура проєкту

```
shop_api_project/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   └── shop/
│       ├── __init__.py
│       ├── api.py            # FastAPI application, endpoints
│       ├── schemas.py        # Pydantic schemas
│       ├── database.py       # engine, SessionLocal, Base
│       ├── models.py         # Category, Product, Order
│       ├── repositories.py   # Repository layer
│       ├── services.py       # Service layer, транзакції
│       ├── dependencies.py   # get_session, get_service, get_http_client
│       ├── external_api.py   # HTTPX, Task, gather, Semaphore, retry
│       ├── exceptions.py     # доменні exceptions → HTTP status codes
│       ├── config.py         # налаштування з environment
│       └── seed.py           # демонстраційні дані
├── docs/screenshots/         # Swagger UI та виконання endpoints
└── tests/
    ├── conftest.py
    ├── test_api.py
    ├── test_async.py
    ├── test_config.py
    └── test_seed.py
```

4. Опис REST API

API побудовано навколо ресурсів products і categories. Запит клієнта проходить через FastAPI, де тіло запиту автоматично перевіряється Pydantic-схемами (помилка валідації → 422). Endpoint викликає Service Layer, який реалізує бізнес-правила й керує транзакціями (commit/rollback). Service Layer працює з БД лише через Repository Layer (SQLAlchemy). Доменні exceptions (наприклад, ProductNotFoundError) перетворюються на HTTP-відповіді єдиним обробником із структурованим JSON {"detail": ..., "code": ...}. Комунікація stateless, формат даних — JSON.

Для БД використано SQLAlchemy 2.x та SQLite (таблиці categories, products, orders). Покупка товару виконується в одній транзакції: зменшення залишку та створення запису в orders або фіксуються разом, або повністю скасовуються (rollback).

5. Список endpoints

Method	URL	Статуси	Призначення
GET	/	200	Службова інформація про API
GET	/categories	200	Список категорій
POST	/categories	201, 409, 422	Створення категорії
GET	/categories/{category_id}/products	200, 404	Товари категорії
GET	/products	200, 400, 422	Список товарів: пошук, фільтри, сортування, пагінація
POST	/products	201, 404, 422	Додавання товару
GET	/products/{product_id}	200, 404, 422	Отримання товару за id
PATCH	/products/{product_id}	200, 400, 404, 422	Зміна ціни, кількості, назви
DELETE	/products/{product_id}	204, 404	Видалення товару
POST	/products/{product_id}/purchase	200, 404, 409, 422	Транзакційна покупка товару
GET	/products/summary	200	Вартість залишків і найдорожчий товар
POST	/products/batch	200, 404, 502, 503, 504	Паралельне отримання кількох товарів (async)

6. Таблиця HTTP methods

Method	Призначення	CRUD	Використання в проєкті
GET	отримання даних	Read	GET /products, GET /products/{id}, GET /categories/{id}/products
POST	створення / дія	Create	POST /products, POST /categories, POST /products/{id}/purchase
PUT	повне оновлення	Update	не використовується (замість нього PATCH)
PATCH	часткове оновлення	Update	PATCH /products/{id}: змінюється лише передане поле
DELETE	видалення	Delete	DELETE /products/{id} → 204 No Content

7. Приклади path parameters

Path parameter — частина URL, що ідентифікує ресурс. Тип перевіряється автоматично: для GET /products/abc FastAPI повертає 422.

```
GET    /products/1
PATCH  /products/2
DELETE /products/3
POST   /products/1/purchase
GET    /categories/1/products
```

```
@app.get("/products/{product_id}", response_model=ProductResponse, tags=["products"])
async def get_product(product_id: int, service: ShopService = Depends(get_service)):
    return service.get_product(product_id)
```

8. Приклади query parameters

Query parameters задають пошук, фільтри, сортування та пагінацію в GET /products. Усі мають валідацію (Query(ge=..., le=...)).

```
GET /products?q=lap
GET /products?category_id=1&sort_by=price&descending=true
GET /products?min_price=100&max_price=1000
GET /products?limit=10&offset=20
```

```
@app.get("/products", response_model=list[ProductResponse], tags=["products"])
async def list_products(
    q: str | None = Query(default=None, min_length=1, description="Пошук за назвою"),
    category_id: int | None = Query(default=None, gt=0),
    min_price: float | None = Query(default=None, ge=0),
    max_price: float | None = Query(default=None, ge=0),
    sort_by: Literal["name", "price", "quantity"] = "name",
    descending: bool = False,
    limit: int = Query(default=50, ge=1, le=100),
    offset: int = Query(default=0, ge=0),
    service: ShopService = Depends(get_service),
):
    return service.list_products(
        q=q, category_id=category_id, min_price=min_price, max_price=max_price,
        sort_by=sort_by, descending=descending, limit=limit, offset=offset,
    )
```

9. Pydantic schemas

```
class ProductCreate(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    category_id: int = Field(gt=0)
    price: float = Field(gt=0)
    quantity: int = Field(ge=0)



class ProductUpdate(BaseModel):
    name: str | None = Field(default=None, min_length=2, max_length=100)
    price: float | None = Field(default=None, gt=0)
    quantity: int | None = Field(default=None, ge=0)
```

```
class ProductResponse(BaseModel):
    id: int
    name: str
    category_id: int
    price: float
    quantity: int
    model_config = {"from_attributes": True}


class PurchaseRequest(BaseModel):
    quantity: int = Field(gt=0, le=1000)


class PurchaseResponse(BaseModel):
    order_id: int
    product_id: int
    quantity: int
    total_price: float
    remaining_quantity: int
```

10. Validation rules

Поле	Правило	Результат при порушенні
name	рядок, 2–100 символів	422 Unprocessable Entity
category_id	ціле, > 0, категорія існує	422 (формат) / 404 (категорія не знайдена)
price	число, > 0	422
quantity (товар)	ціле, ≥ 0	422
quantity (покупка)	ціле, 1–1000, не більше залишку	422 / 409 Conflict (недостатньо товару)
PATCH body	хоча б одне поле	400 Bad Request (empty_update)
min_price / max_price	≥ 0, min ≤ max	422 / 400 (invalid_price_range)
limit / offset	1–100 / ≥ 0	422
sort_by	name | price | quantity	422
product_ids (batch)	список 1–50 цілих	422
category.name	унікальне	409 Conflict

11. Код FastAPI application

Файл api.py: lifespan створює таблиці та один спільний httpx.AsyncClient на весь час роботи застосунку (lifecycle management), закриваючи його при зупинці. Єдиний exception handler перетворює доменні помилки на структуровані JSON-відповіді.

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    """Create tables and one shared AsyncClient for the whole application."""
    database.init_db()
    settings = Settings.from_env()
    app.state.http_client = httpx.AsyncClient(
```

```
        base_url=settings.api_base_url,
        timeout=httpx.Timeout(5.0, connect=2.0),
    )
    try:
        yield
    finally:
        await app.state.http_client.aclose()



app = FastAPI(
    title="Shop API",
    version="0.8.0",
    description="Асинхронний REST API інтернет-магазину (лабораторна робота №8).",
    lifespan=lifespan,
)



@app.exception_handler(ShopError)
async def shop_error_handler(request: Request, exc: ShopError) -> JSONResponse:
    return JSONResponse(
        status_code=exc.status_code,
        content={"detail": str(exc), "code": exc.code},
    )
```

12. Реалізація GET

```
@app.get("/products/{product_id}", response_model=ProductResponse, tags=["products"])
async def get_product(product_id: int, service: ShopService = Depends(get_service)):
    return service.get_product(product_id)
```

```
@app.get(
    "/categories/{category_id}/products",
    response_model=list[ProductResponse],
    tags=["categories"],
)
async def category_products(category_id: int, service: ShopService = Depends(get_service)):
    return service.products_in_category(category_id)
```

13. Реалізація POST

```
    "/products",
    response_model=ProductResponse,
    status_code=status.HTTP_201_CREATED,
    tags=["products"],
)
async def create_product(data: ProductCreate, service: ShopService = Depends(get_service)):
    return service.create_product(data)
```

Бізнес-логіка в Service Layer:

```
    def create_product(self, data: ProductCreate) -> Product:
        self._require_category(data.category_id)
        product = self.products.add(Product(**data.model_dump()))
        self.session.commit()
```

```
        return product
```

14. Реалізація PATCH

```
@app.patch("/products/{product_id}", response_model=ProductResponse, tags=["products"])
async def update_product(
    product_id: int, data: ProductUpdate, service: ShopService = Depends(get_service)
):
    return service.update_product(product_id, data)
```

```
    def update_product(self, product_id: int, data: ProductUpdate) -> Product:
        changes = data.model_dump(exclude_unset=True)
        if not changes:
            raise EmptyUpdateError("No fields to update")
        product = self.get_product(product_id)
        for field, value in changes.items():
            setattr(product, field, value)
        self.session.commit()
        return product
```

model_dump(exclude_unset=True) дозволяє відрізнити «поле не передано» від «поле передано зі значенням», тому змінюються лише передані поля.

15. Реалізація DELETE

```
@app.delete(
    "/products/{product_id}", status_code=status.HTTP_204_NO_CONTENT, tags=["products"]
)
async def delete_product(product_id: int, service: ShopService = Depends(get_service)) -> None:
    service.delete_product(product_id)
```

16. Транзакційна покупка товару

```
    # ---- transaction -------------------------------------------------------
    def purchase(self, product_id: int, quantity: int) -> PurchaseResponse:
        """Atomically decrease stock and create an order (all or nothing)."""
        try:
            product = self.get_product(product_id)
            if product.quantity < quantity:
                raise InsufficientStockError(
                    f"Only {product.quantity} item(s) left, requested {quantity}"
                )
            product.quantity -= quantity
            order = self.orders.add(
                Order(
                    product_id=product.id,
                    quantity=quantity,
                    total_price=round(product.price * quantity, 2),
                )
            )
            self.session.commit()
        except Exception:
            self.session.rollback()
            raise
```

```
        return PurchaseResponse(
            order_id=order.id,
            product_id=product.id,
            quantity=quantity,
            total_price=order.total_price,
            remaining_quantity=product.quantity,
        )
```

Зменшення залишку та створення Order фіксуються однією командою commit(). Якщо виникає будь-яка помилка (недостатній залишок, збій під час збереження замовлення), виконується rollback() і стан БД не змінюється. Це перевіряють тести test_purchase_more_than_stock_returns_409_and_keeps_stock та test_purchase_is_rolled_back_when_order_cannot_be_saved.

17. Приклад JSON request

```
POST /products
Content-Type: application/json

{
  "name": "Laptop",
  "category_id": 1,
  "price": 25000,
  "quantity": 3
}
```

```
POST /products/1/purchase
Content-Type: application/json

{
  "quantity": 2
}
```

18. Приклад JSON response

Відповіді нижче отримано з реального запуску uvicorn (повний журнал — у docs/live_requests_log.txt).

```
201 Created
{
  "id": 1,
  "name": "Laptop",
  "category_id": 1,
  "price": 25000.0,
  "quantity": 3
}
```

```
200 OK  (POST /products/1/purchase)
{
  "order_id": 1,
  "product_id": 1,
  "quantity": 2,
  "total_price": 50000.0,
  "remaining_quantity": 1
}
```

```
409 Conflict  (POST /products/1/purchase, quantity = 5)
{
  "detail": "Only 1 item(s) left, requested 5",
  "code": "insufficient_stock"
}
```

```
404 Not Found  (GET /products/999)
{
  "detail": "Product 999 not found",
  "code": "product_not_found"
}
```

```
422 Unprocessable Entity  (POST /products, name = "X", price = -10)
{"detail": [
  {"type": "string_too_short", "loc": ["body", "name"], "msg": "String should have at least 2
characters", ...},
  {"type": "greater_than", "loc": ["body", "price"], "msg": "Input should be greater than 0", ...}]}
```

```
GET /products/summary  → 200 OK
{
  "products_count": 3,
  "total_stock_value": 40300.0,
  "most_expensive": {"id": 1, "name": "Laptop", "category_id": 1, "price": 25000.0, "quantity": 1}
}
```

19. Приклад async/await

Усі endpoints оголошено як async def. Функція fetch_product виконує неблокуючий HTTP-запит: await віддає керування event loop, доки очікується відповідь.

```
async def fetch_product(client: httpx.AsyncClient, product_id: int) -> dict:
    """Single GET request to the REST API."""
    response = await client.get(f"/products/{product_id}")
    response.raise_for_status()
    return response.json()
```

20. Приклад asyncio.Task

Кожен запит обгортається в asyncio.Task через create_task(): Task починає виконуватися одразу, паралельно з іншими.

```
    tasks = [
        asyncio.create_task(
            limited_fetch(client, semaphore, pid, attempts=attempts, backoff_base=backoff_base)
        )
        for pid in product_ids
    ]
```

21. Приклад asyncio.gather

```
async def fetch_many(
    client: httpx.AsyncClient,
    product_ids: list[int],
    *,
```

```
    max_concurrency: int = 5,
    attempts: int = 3,
    backoff_base: float = 0.2,
) -> list[dict]:
    """Fetch several products in parallel (Task + gather + Semaphore)."""
    semaphore = asyncio.Semaphore(max_concurrency)
    tasks = [
        asyncio.create_task(
            limited_fetch(
                client, semaphore, pid, attempts=attempts, backoff_base=backoff_base
            )
        )
        for pid in product_ids
    ]
    try:
        return list(await asyncio.gather(*tasks))
    except BaseException:
        for task in tasks:
            task.cancel()
        await asyncio.gather(*tasks, return_exceptions=True)
        raise
```

gather() чекає завершення всіх Tasks і повертає результати в порядку запитів. Якщо один Task завершується помилкою, решту скасовано (cancel), щоб не залишати «висячих» запитів; це перевіряє test_failed_task_cancels_the_other_tasks.

22. Приклад timeout

Є два рівні timeout: httpx.Timeout(5.0, connect=2.0) для окремого HTTP-запиту (створюється в lifespan) та asyncio.wait_for(..., timeout=settings.batch_timeout) для всієї batch-операції. При перевищенні ліміту endpoint повертає 504 Gateway Timeout.

```
@app.post("/products/batch", response_model=list[ProductResponse], tags=["async"])
async def batch_products(
    body: BatchRequest,
    client: httpx.AsyncClient = Depends(get_http_client),
    settings: Settings = Depends(get_settings),
):
    """Fetch several products in parallel through HTTPX (Task, gather, Semaphore, timeout, retry)."""
    try:
        return await asyncio.wait_for(
            fetch_many(
                client,
                body.product_ids,
                max_concurrency=settings.max_concurrency,
                attempts=settings.retry_attempts,
                backoff_base=settings.backoff_base,
            ),
            timeout=settings.batch_timeout,
        )
    except asyncio.TimeoutError:
        raise HTTPException(status.HTTP_504_GATEWAY_TIMEOUT, "Batch operation timed out")
    except httpx.HTTPStatusError as exc:
        if exc.response.status_code == 404:
            raise HTTPException(status.HTTP_404_NOT_FOUND, "One of the products was not found")
        raise HTTPException(status.HTTP_502_BAD_GATEWAY, "Upstream request failed")
    except httpx.RequestError:
        raise HTTPException(status.HTTP_503_SERVICE_UNAVAILABLE, "Upstream is unavailable")
```

23. Приклад retry

Retry з exponential backoff (0.2 с, 0.4 с, 0.8 с …). Повторюються лише мережеві помилки та відповіді 5xx; помилки клієнта (наприклад, 404) повторювати немає сенсу.

```
def _is_retryable(exc: Exception) -> bool:
    """Network errors and 5xx are retried; 4xx (e.g. 404) are not."""
    if isinstance(exc, httpx.HTTPStatusError):
        return exc.response.status_code >= 500
    return isinstance(exc, httpx.RequestError)



async def fetch_with_retry(
    client: httpx.AsyncClient,
    product_id: int,
    *,
    attempts: int = 3,
    backoff_base: float = 0.2,
) -> dict:
    """Retry with exponential backoff: 0.2 s, 0.4 s, 0.8 s ..."""
    for attempt in range(1, attempts + 1):
        try:
            return await fetch_product(client, product_id)
        except (httpx.RequestError, httpx.HTTPStatusError) as exc:
            if attempt == attempts or not _is_retryable(exc):
                raise
            await asyncio.sleep(backoff_base * 2 ** (attempt - 1))
    raise RuntimeError("Unexpected retry state")  # pragma: no cover
```

24. Приклад Semaphore

```
async def limited_fetch(
    client: httpx.AsyncClient,
    semaphore: asyncio.Semaphore,
    product_id: int,
    *,
    attempts: int = 3,
    backoff_base: float = 0.2,
) -> dict:
    """Semaphore limits the number of simultaneous requests."""
    async with semaphore:
        return await fetch_with_retry(
            client, product_id, attempts=attempts, backoff_base=backoff_base
        )
```

Semaphore(max_concurrency) дозволяє одночасно виконувати не більше max_concurrency запитів (за замовчуванням 5, змінюється через SHOP_MAX_CONCURRENCY). Тест test_semaphore_limits_concurrency запускає 10 запитів із лімітом 3 і перевіряє, що пікова кількість одночасних запитів дорівнює 3.

25. Приклад HTTPX

Для HTTP-запитів використано httpx.AsyncClient. Клієнт створюється один раз у lifespan і передається в endpoint через dependency get_http_client, а не створюється на кожен запит (повторне використання з’єднань).

```
def get_http_client(request: Request) -> httpx.AsyncClient:
```

```
    """Shared AsyncClient created in the application lifespan."""
    return request.app.state.http_client
```

У тестах той самий код працює без мережі: AsyncClient використовує ASGITransport(app=app) для звернень до самого застосунку, а для перевірки retry/timeout — httpx.MockTransport.

```
    transport = ASGITransport(app=app, raise_app_exceptions=False)
    async with AsyncClient(transport=transport, base_url="http://test") as http:
        app.dependency_overrides[get_session] = override_session
        app.dependency_overrides[get_http_client] = lambda: http
        app.dependency_overrides[get_settings] = lambda: Settings(
            batch_timeout=2.0, max_concurrency=3, retry_attempts=2, backoff_base=0.0
        )
```

26. Результати API tests

Файл	Тестів	Що перевіряє
tests/test_api.py	47	статус-коди 200/201/204/400/404/409/422, JSON-відповіді, validation, фільтри, пошук, сортування, пагінація, PATCH, DELETE, summary, транзакційна покупка та rollback
tests/test_async.py	17	gather (порядок результатів, паралельність), Semaphore, retry (успіх після збоїв, вичерпання спроб, 4xx без retry, мережеві помилки), скасування Tasks, batch endpoint (200/404/422/502/503/504), lifespan
tests/test_config.py	2	налаштування з environment (monkeypatch)
tests/test_seed.py	1	ідемпотентність seed
Разом	67	

Приклад API-тесту (створення, validation, асинхронний HTTPX-клієнт через ASGITransport):

```
async def test_create_product_returns_201_and_json(client, category):
    payload = {"name": "Laptop", "category_id": category["id"], "price": 25000, "quantity": 3}
    response = await client.post("/products", json=payload)
    assert response.status_code == 201
    assert response.json() == {"id": 1, "name": "Laptop", "category_id": 1, "price": 25000.0,
"quantity": 3}



@pytest.mark.parametrize(
    "patch",
    [
        {"price": 0},
        {"price": -5},
        {"quantity": -1},
        {"name": "A"},
        {"name": ""},
        {"category_id": 0},
        {"price": "abc"},
    ],
    ids=["zero-price", "negative-price", "negative-quantity", "short-name", "empty-name", "bad-
category", "price-not-number"],
)
async def test_create_product_validation_errors_return_422(client, category, patch):
    payload = {"name": "Laptop", "category_id": category["id"], "price": 10, "quantity": 1, **patch}
    response = await client.post("/products", json=payload)
    assert response.status_code == 422
```

```
async def test_purchase_more_than_stock_returns_409_and_keeps_stock(client, laptop, db_session):
    response = await client.post(f"/products/{laptop['id']}/purchase", json={"quantity": 4})
    assert response.status_code == 409
    assert response.json()["code"] == "insufficient_stock"
    assert (await client.get(f"/products/{laptop['id']}")).json()["quantity"] == 3
    assert OrderRepository(db_session).count() == 0



@pytest.mark.parametrize("quantity", [0, -1, 1001, "x"])
```

Приклад тесту retry (MockTransport імітує тимчасові збої 503):

```
async def test_retry_succeeds_after_temporary_failures():
    calls = 0

    def flaky(request):
        nonlocal calls
        calls += 1
        return httpx.Response(503) if calls < 3 else ok(request)


    async with mock_client(flaky) as client:
        result = await fetch_with_retry(client, 7, attempts=3, backoff_base=0)
    assert result == {"id": 7}
    assert calls == 3
```

27. Screenshot Swagger UI

Swagger UI (http://127.0.0.1:8000/docs) формується автоматично з OpenAPI-опису. Видно endpoints, згруповані за тегами (service, categories, products, async), і Pydantic schemas.

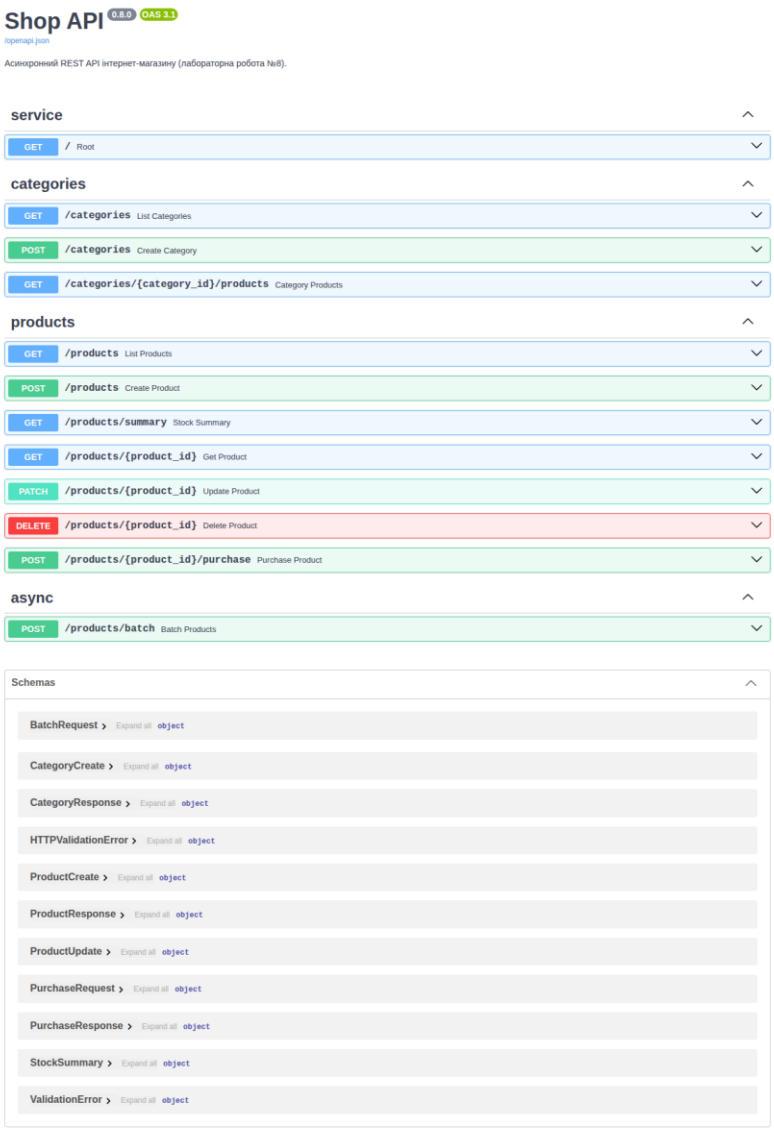


Рис. 1. Swagger UI: список endpoints і schemas

28. Screenshots виконання endpoints

Нижче — запити, виконані через кнопку «Try it out» у Swagger UI до працюючого сервера uvicorn.

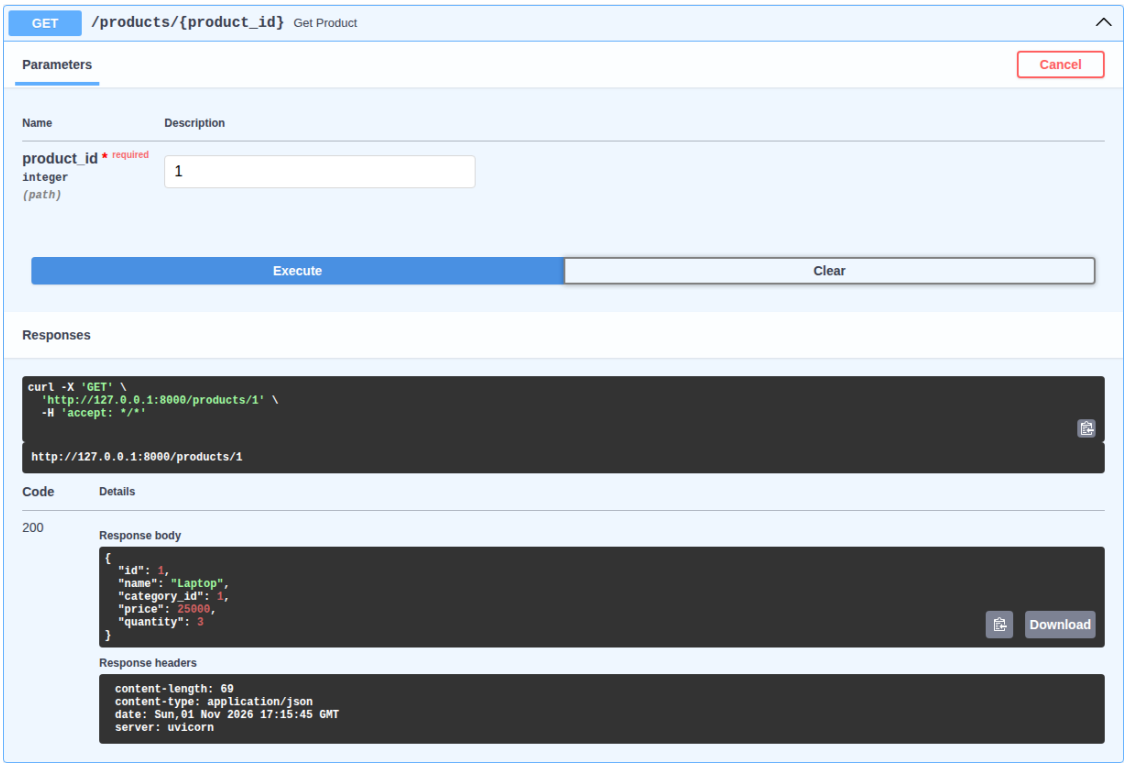


Рис. 2. GET /products/{product_id} → 200 OK

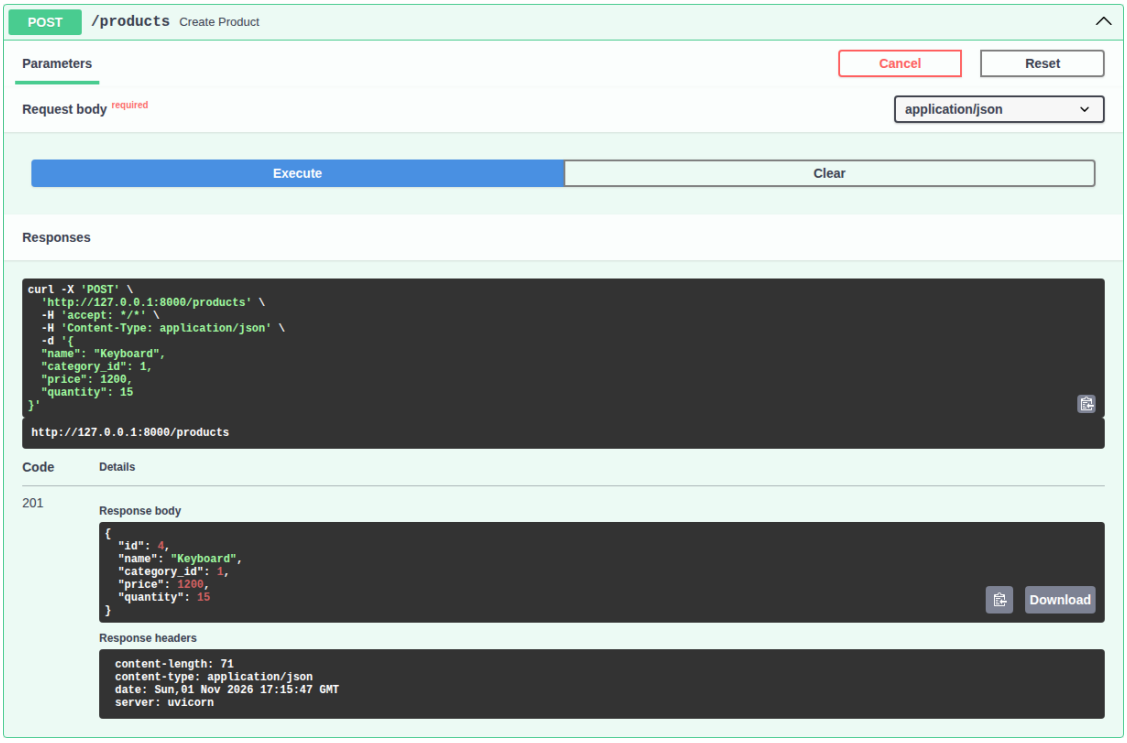


Рис. 3. POST /products → 201 Created

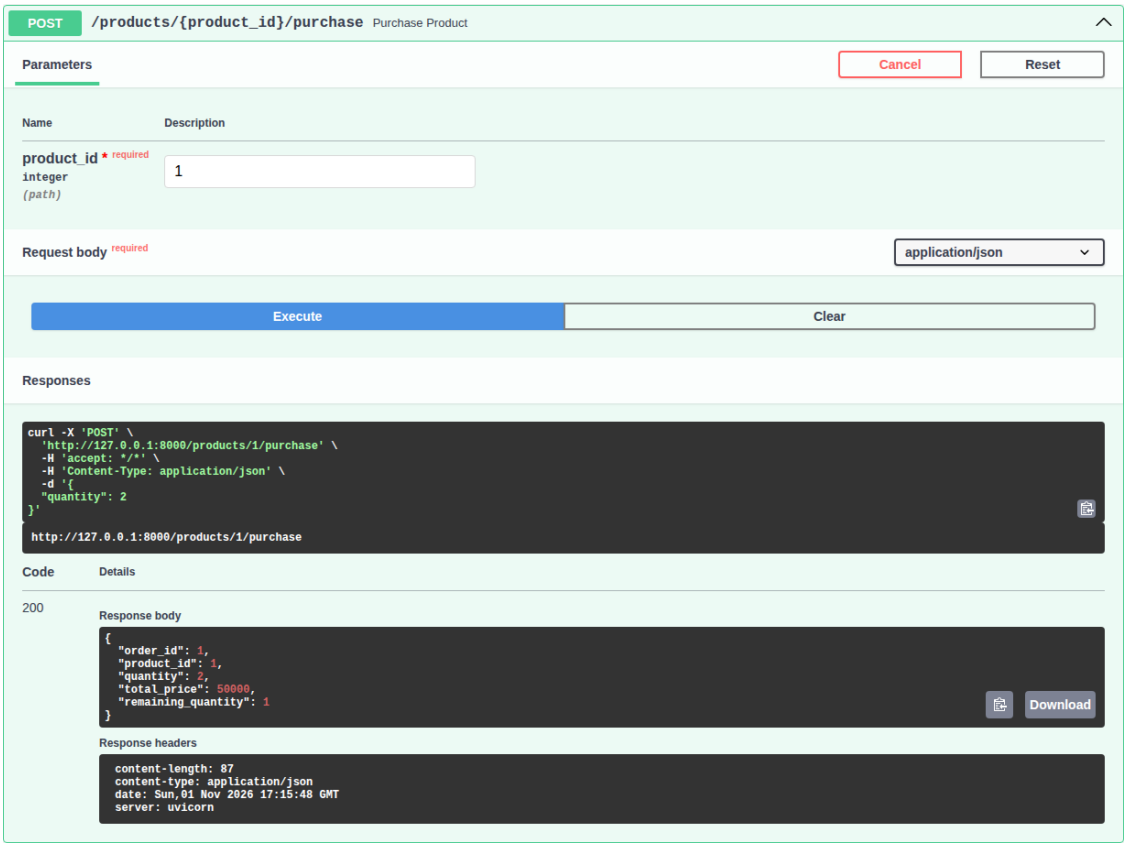


Рис. 4. POST /products/{product_id}/purchase → 200 OK (транзакційна покупка)

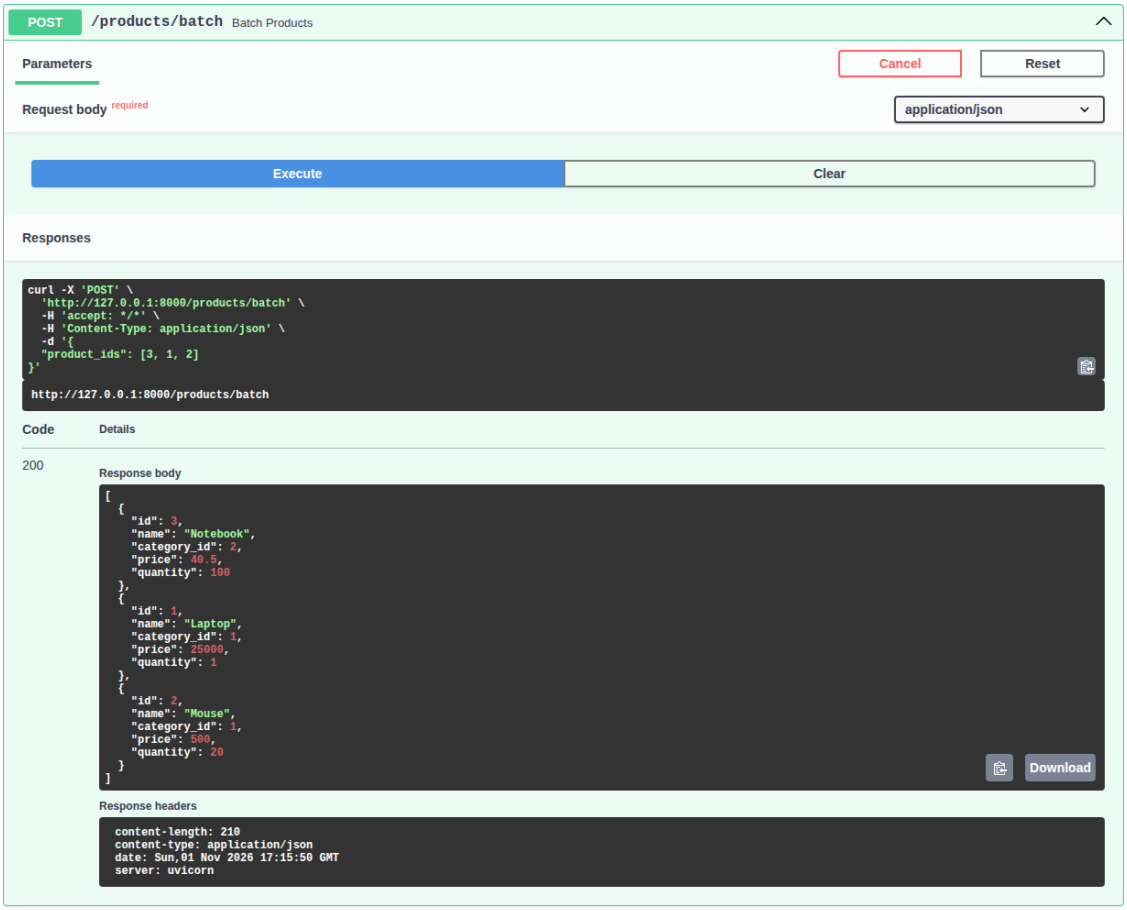


Рис. 5. POST /products/batch → 200 OK (паралельне отримання через HTTPX)

29. Результати pytest -v

```
python -m pytest -v --cov=shop --cov-branch --cov-report=term-missing
```

```
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/shop_api_project
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0, asyncio-1.4.1, anyio-4.16.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=function,
asyncio_default_test_loop_scope=function
collecting ... collected 67 items
test_api.py::test_root  PASSED
test_api.py::test_create_category_returns_201  PASSED
test_api.py::test_duplicate_category_returns_409  PASSED
test_api.py::test_list_categories  PASSED
test_api.py::test_category_products  PASSED
test_api.py::test_category_products_unknown_category_returns_404  PASSED
test_api.py::test_create_product_returns_201_and_json  PASSED
test_api.py::test_create_product_validation_errors_return_422[zero-price]  PASSED
test_api.py::test_create_product_validation_errors_return_422[negative-price]  PASSED
test_api.py::test_create_product_validation_errors_return_422[negative-quantity]  PASSED
test_api.py::test_create_product_validation_errors_return_422[short-name]  PASSED
test_api.py::test_create_product_validation_errors_return_422[empty-name]  PASSED
test_api.py::test_create_product_validation_errors_return_422[bad-category]  PASSED
test_api.py::test_create_product_validation_errors_return_422[price-not-number]  PASSED
test_api.py::test_create_product_missing_field_returns_422  PASSED
```

```
test_api.py::test_create_product_unknown_category_returns_404  PASSED
test_api.py::test_get_product  PASSED
test_api.py::test_get_missing_product_returns_404  PASSED
test_api.py::test_get_product_with_non_integer_id_returns_422  PASSED
test_api.py::test_list_products_sorted_by_name_by_default  PASSED
test_api.py::test_filter_by_category  PASSED
test_api.py::test_search_by_name_is_case_insensitive  PASSED
test_api.py::test_filter_by_price_range  PASSED
test_api.py::test_invalid_price_range_returns_400  PASSED
test_api.py::test_sort_by_price_descending  PASSED
test_api.py::test_pagination  PASSED
test_api.py::test_invalid_query_parameters_return_422[params0]  PASSED
test_api.py::test_invalid_query_parameters_return_422[params1]  PASSED
test_api.py::test_invalid_query_parameters_return_422[params2]  PASSED
test_api.py::test_invalid_query_parameters_return_422[params3]  PASSED
test_api.py::test_patch_price_and_quantity  PASSED
test_api.py::test_patch_with_empty_body_returns_400  PASSED
test_api.py::test_patch_invalid_value_returns_422  PASSED
test_api.py::test_patch_missing_product_returns_404  PASSED
test_api.py::test_delete_product_returns_204_then_404  PASSED
test_api.py::test_delete_missing_product_returns_404  PASSED
test_api.py::test_summary_total_value_and_most_expensive  PASSED
test_api.py::test_summary_of_empty_shop  PASSED
test_api.py::test_purchase_decreases_stock_and_creates_order  PASSED
test_api.py::test_purchase_all_remaining_stock_is_allowed  PASSED
test_api.py::test_purchase_more_than_stock_returns_409_and_keeps_stock  PASSED
test_api.py::test_purchase_invalid_quantity_returns_422[0]  PASSED
test_api.py::test_purchase_invalid_quantity_returns_422[-1]  PASSED
test_api.py::test_purchase_invalid_quantity_returns_422[1001]  PASSED
test_api.py::test_purchase_invalid_quantity_returns_422[x]  PASSED
test_api.py::test_purchase_unknown_product_returns_404  PASSED
test_api.py::test_purchase_is_rolled_back_when_order_cannot_be_saved  PASSED
test_async.py::test_fetch_many_returns_results_in_request_order  PASSED
test_async.py::test_fetch_many_runs_requests_in_parallel  PASSED
test_async.py::test_semaphore_limits_concurrency  PASSED
test_async.py::test_retry_succeeds_after_temporary_failures  PASSED
test_async.py::test_retry_gives_up_after_all_attempts  PASSED
test_async.py::test_client_errors_are_not_retried  PASSED
test_async.py::test_network_errors_are_retried  PASSED
test_async.py::test_failed_task_cancels_the_other_tasks  PASSED
test_async.py::test_batch_endpoint_fetches_products_through_httpx  PASSED
test_async.py::test_batch_endpoint_returns_404_for_missing_product  PASSED
test_async.py::test_batch_endpoint_validation[empty]  PASSED
test_async.py::test_batch_endpoint_validation[too-many]  PASSED
test_async.py::test_batch_endpoint_validation[not-int]  PASSED
test_async.py::test_batch_endpoint_timeout_returns_504  PASSED
test_async.py::test_batch_endpoint_maps_upstream_failures[upstream-unavailable]  PASSED
test_async.py::test_batch_endpoint_maps_upstream_failures[upstream-500]  PASSED
test_async.py::test_lifespan_creates_and_closes_http_client  PASSED
test_config.py::test_defaults  PASSED
test_config.py::test_settings_from_environment  PASSED
test_seed.py::test_seed_is_idempotent  PASSED
```

```
Name                       Stmts   Miss Branch BrPart  Cover   Missing
----------------------------------------------------------------------
src/shop/__init__.py           1      0      0      0   100%
src/shop/api.py               71      0      2      0   100%
src/shop/config.py            15      0      0      0   100%
src/shop/database.py          15      0      0      0   100%
src/shop/dependencies.py      19      6      0      0    68%   16-20, 28, 33
src/shop/exceptions.py        21      0      0      0   100%
src/shop/external_api.py      32      0      6      0   100%
src/shop/models.py            23      0      0      0   100%
src/shop/repositories.py      60      0      8      0   100%
src/shop/schemas.py           38      0      0      0   100%
src/shop/seed.py              16      1      4      1    90%   25
src/shop/services.py          71      0     14      0   100%
```

```
---------------------------------------------------------------------
TOTAL                        382      7     34      1    98%
Coverage HTML written to dir htmlcov
============================== 67 passed in 2.25s ==============================
```

Показник	Значення	Коментар
Зібрано tests	67	
Passed	67	
Failed	0	
Покриття (statement + branch)	98 %	dependencies.py 68 % — реальні get_session/get_settings у тестах підміняються (dependency_overrides)

30. Робота з Git

```
git init
git add .gitignore pyproject.toml README.md
src/shop/{__init__,config,database,models,exceptions,dependencies,api}.py
git commit -m "Add FastAPI application"
git add src/shop/schemas.py
git commit -m "Add Pydantic API schemas"
git add src/shop/repositories.py src/shop/services.py src/shop/seed.py
git commit -m "Implement REST endpoints logic: repositories and services"
git add src/shop/external_api.py
git commit -m "Add async HTTP client with concurrent operations and retry logic"
git add tests
git commit -m "Add API integration tests"
git add docs
git commit -m "Add Swagger screenshots and live request log"
```

30.1. Результат git status

```
On branch master
nothing to commit, working tree clean
```

30.2. Результат git log --oneline

```
9abd24b Add Swagger screenshots and live request log
3d4db13 Add API integration tests
f50655c Add async HTTP client with concurrent operations and retry logic
1b39abd Implement REST endpoints logic: repositories and services
0daaf7c Add Pydantic API schemas
3a9ac5b Add FastAPI application
```

31. Особливості реалізації

1) Persistence layer. Для БД використано SQLAlchemy + SQLite із таблицями categories, products, orders та шарами Repository/Service. Транзакційну покупку реалізовано через commit/rollback у Service Layer, як вимагає опис варіанта.

2) Синхронна робота з БД в async-endpoints. Endpoints оголошені як async def, а запити SQLAlchemy синхронні й короткі (SQLite). У production-системі для навантажених сервісів доцільно перейти на AsyncSession або виконувати блокуючі виклики в thread pool.

3) Маршрут /products/summary оголошено перед /products/{product_id}, щоб слово «summary» не інтерпретувалося як product_id.

4) Batch-endpoint звертається до API через HTTPX за адресою SHOP_API_BASE_URL (за замовчуванням http://127.0.0.1:8000), тобто демонструє реальний HTTP-запит; у тестах цей же код працює через ASGITransport.

5) Screenshots Swagger UI зроблено в headless-браузері з тими самими файлами swagger-ui, які FastAPI за замовчуванням підвантажує з CDN; вигляд сторінки /docs ідентичний.

32. Відповіді на контрольні запитання

1. Що таке комп'ютерна мережа?
Сукупність пристроїв, з'єднаних каналами зв'язку для обміну даними.

2. Що таке клієнт-серверна архітектура?
Модель, у якій клієнт надсилає запити, а сервер їх обробляє й повертає відповіді.

3. Що таке HTTP?
HyperText Transfer Protocol — протокол прикладного рівня для обміну даними між клієнтом і сервером за схемою «запит — відповідь».

4. Що містить HTTP request?
Method, URL, headers і (необов'язково) body.

5. Що містить HTTP response?
Status code, headers і body (наприклад, JSON).

6. Для чого використовується GET?
Для отримання даних без їх зміни.

7. Для чого використовується POST?
Для створення ресурсу або виконання дії (у проєкті — створення товару, покупка).

8. Для чого використовується PUT?
Для повної заміни ресурсу.

9. Для чого використовується PATCH?
Для часткового оновлення ресурсу.

10. Для чого використовується DELETE?
Для видалення ресурсу.

11. Що означає status code 200?
OK — запит виконано успішно.

12. Що означає 201?
Created — ресурс створено.

13. Що означає 204?

No Content — успішно, але тіла відповіді немає (типово для DELETE).

14. Що означає 404?

Not Found — ресурс не знайдено.

15. Що означає 422?

Unprocessable Entity — дані не пройшли валідацію.

16. Що таке JSON?

JavaScript Object Notation — текстовий формат обміну структурованими даними (об’єкти, масиви, рядки, числа).

17. Що таке REST?

Архітектурний стиль API, побудований на ресурсах, HTTP methods, status codes і stateless-взаємодії.

18. Що таке resource у REST?

Сутність, що має URL і над якою виконуються операції (products, categories).

19. Що таке endpoint?

Конкретний URL разом із HTTP method, що обробляє певний тип запитів.

20. Що таке path parameter?

Змінна частина шляху URL, наприклад /products/{product_id}.

21. Що таке query parameter?

Параметр після «?» в URL (фільтри, сортування, пагінація): /products?limit=10.

22. Що таке request body?

Дані, що передаються в тілі запиту (зазвичай JSON) у POST/PUT/PATCH.

23. Що таке FastAPI?

Сучасний async-фреймворк для створення API на Python із автоматичною валідацією та документацією (OpenAPI).

24. Для чого використовується Uvicorn?

ASGI-сервер, що запускає FastAPI-застосунок і приймає HTTP-запити.

25. Для чого потрібний Swagger UI?

Інтерактивна документація (/docs), де можна переглядати й виконувати запити до API.

26. Що таке Pydantic?

Бібліотека для опису моделей даних і їх валідації на основі type hints.

27. Що таке validation?

Перевірка коректності вхідних даних (типи, діапазони, довжини) до виконання бізнес-логіки.

28. Що таке response model?

Схема відповіді (response_model), що визначає й фільтрує поля, які повертає endpoint.

29. Що таке synchronous programming?

Операції виконуються послідовно: наступна починається після завершення попередньої, очікування блокує потік.

30. Що таке asynchronous programming?

Під час очікування (I/O) керування передається іншим завданням, тому багато операцій виконуються конкурентно в одному потоці.

31. Для чого використовується async?

Для оголошення coroutine-функції (async def), яку можна призупиняти на await.

32. Для чого використовується await?

Для очікування результату coroutine/Task із віддачею керування event loop.

33. Що таке coroutine?

Об’єкт, що повертається викликом async def; виконується лише під керуванням event loop (через await або Task).

34. Що таке event loop?

Цикл, що планує й запускає coroutines/Tasks та перемикає їх під час очікування I/O.

35. Що таке Task?

Обгортка над coroutine, яка планується на виконання в event loop одразу після створення.

36. Для чого використовується asyncio.create_task()?

Щоб запустити coroutine паралельно як Task, не очікуючи її завершення одразу.

37. Для чого використовується asyncio.gather()?

Щоб дочекатися кількох awaitable одночасно й отримати список результатів у порядку передавання.

38. Що таке timeout?

Максимальний час очікування операції; після його перевищення операція переривається (asyncio.wait_for, httpx.Timeout).

39. Для чого потрібний retry?

Для повторної спроби запиту після тимчасових збоїв (мережа, 5xx), що підвищує надійність.

40. Що таке backoff?

Збільшення паузи між повторними спробами (часто експоненційно: 0.2 с, 0.4 с, 0.8 с), щоб не перевантажувати сервіс.

41. Для чого використовується Semaphore?

Для обмеження кількості одночасних операцій (рівня паралелізму).

42. Що таке HTTPX?

Сучасна бібліотека HTTP-клієнта для Python із підтримкою синхронного й асинхронного API.

43. Чим httpx.AsyncClient відрізняється від синхронного клієнта?

AsyncClient має async-методи (await client.get), не блокує event loop й дозволяє робити багато запитів паралельно.

44. Чому AsyncClient бажано повторно використовувати?

Він тримає пул з’єднань: повторне використання уникає витрат на встановлення з’єднань і економить ресурси.

45. Навіщо обмежувати паралелізм?

Щоб не перевантажити сервер, мережу, БД й не перевищити ліміти зовнішніх сервісів.

46. Чому не слід запускати тисячі tasks одночасно?

Це вичерпує пам’ять, з’єднання й дескриптори та може призвести до відмов або блокування з боку сервісу; потрібен Semaphore чи пакетна обробка.

47. Як REST API взаємодіє з Repository Layer?

Endpoint викликає Service Layer, який через репозиторії виконує операції з БД; endpoint не працює з БД напряму.

48. Як FastAPI працює з Pydantic?

Параметри й тіло запиту автоматично перетворюються та валідуються за Pydantic-схемами, а відповідь серіалізується за response_model; опис потрапляє в OpenAPI.

49. Як протестувати REST API?

Через TestClient або httpx.AsyncClient з ASGITransport, перевіряючи status code й JSON, з окремою тестовою БД та підміною dependencies.

50. Чим API integration test відрізняється від unit test?

Integration test проходить увесь шлях HTTP → FastAPI → Service → Repository → БД, а unit test перевіряє одну одиницю ізольовано.

33. Висновок

У лабораторній роботі створено асинхронний REST API для інтернет-магазину (варіант №3) на FastAPI. Реалізовано 12 endpoints (GET, POST, PATCH, DELETE) із path і query parameters, Pydantic-схемами та валідацією, правильними HTTP status codes і структурованими помилками. API працює через Service і Repository шари з БД на SQLAlchemy; покупка товару виконується як транзакція з rollback.

Асинхронна частина реалізує паралельне отримання товарів через httpx.AsyncClient із використанням asyncio.Task, gather, Semaphore, timeout (httpx.Timeout та asyncio.wait_for) і retry з exponential backoff. Для API написано 67 автоматизованих тестів (усі passed, покриття 98 %). Отримано практичні навички створення REST API, роботи з HTTP/JSON та асинхронного програмування в Python.